

파이썬 기반 웹 AI 모의면접 플랫폼 구축을 위한 종합 분석 및 상세 설계

1. 서론: 프로젝트 배경 및 문서 개요

1.1 프로젝트의 정의 및 비전

파이썬 기반의 웹 AI 모의면접 플랫폼 구축을 위한 포괄적인 프로젝트 계획 및 상세 설계 문서를 담고 있습니다.

현대 채용 시장에서 기업은 수천 명의 지원자를 효율적으로 검증해야 하는 과제에 직면해 있으며, 구직자는 시간과 장소의 제약 없이 객관적인 피드백을 받을 수 있는 훈련 도구를 필요로 합니다.

본 프로젝트는 이러한 양측의 니즈를 충족시키기 위해, 생성형 AI(Generative AI)와 컴퓨터 비전(Computer Vision), 음성 분석(Audio Analysis) 기술을 융합하여 실제 면접관을 대체하거나 보조할 수 있는 고도화된 AI 에이전트를 개발하는 것을 목표로 합니다.

특히 파이썬(Python) 생태계를 기반으로 한 **FastAPI**와 **LangChain**, **WebRTC** 기술 스택의 통합은 실시간성과 확장성을 동시에 확보하는 전략적 선택입니다.³ 본 문서는 단순한 기획서를 넘어, 개발팀, 데이터 사이언스팀, 인프라팀이 즉시 실행 가능한 수준의 구체적인 명세서(Specification)와 작업 분류 체계(WBS)를 포함합니다.

1.2 문서의 구성 및 범위

본 보고서는 성공적인 프로젝트 수행을 위해 필수적인 7가지 핵심 문서를 상세히 기술합니다.

- 프로젝트 헌장 (Project Charter):** 프로젝트의 목표, 범위, 이해관계자 및 성공 지표 정의.
- 시스템 요구사항 명세서 (SRS):** 기능적, 비기능적 요구사항 및 사용자 페르소나 분석.
- 소프트웨어 아키텍처 설계서 (SAD):** 마이크로서비스 구조, 실시간 데이터 파이프라인, AI 모델 서빙 전략.
- AI 및 데이터 엔지니어링 명세서:** RAG(Retrieval-Augmented Generation) 구조, 감정 분석 및 STT/TTS 파이프라인.
- API 및 데이터 설계서:** 데이터베이스 스키마, 인터페이스 규약(JSON Schema), 통신 프로토콜.
- 작업 분류 체계 (WBS) 및 일정 계획:** 단계별 세부 작업 및 리소스 할당.
- 품질 보증(QA) 및 평가 계획:** 테스트 전략, AI 윤리 및 편향성 검증, 채용 루브릭(Rubric) 설계.

2. 프로젝트 헌장 (Project Charter)

2.1 비즈니스 케이스 및 필요성

채용 과정에서의 병목 현상은 기업의 경쟁력을 저하시키는 주된 요인입니다. 기존의 인간 주도 면접은 일정 조율의 어려움, 면접관의 주관적 편향, 그리고 높은 운영 비용이라는 한계를 가집니다.

AI 모의면접 시스템은 이러한 문제를 해결하기 위해 가용성, 데이터 기반의 객관적 평가, 그리고 대규모 동시 접속 처리를 가능하게 합니다. 특히, 지원자의 비언어적 태도(표정, 목소리 톤)와 언어적 내용(기술적 정확성, 논리력)을 동시에 분석함으로써, 단순한 키워드 매칭을 넘어선 심층적인 역량 평가를 제공합니다.

2.2 프로젝트 목표 (Objectives)

- **실시간성 확보:** 음성 인식부터 AI 응답 생성, 음성 합성까지의 총 지연 시간(End-to-End Latency)을 800ms~1.5s 이내로 단축하여 자연스러운 대화 흐름 구현.
- **평가의 객관화:** 기업별, 직무별 맞춤형 채용 루브릭(Rubric)을 시스템에 통합하여, AI가 정해진 기준에 따라 일관성 있게 점수를 산출하도록 구현.
- **기술적 검증:** 코딩 테스트 환경(IDE)과 시스템 설계용 화이트보드를 통합하여 기술 면접의 실효성 확보.

2.3 주요 이해관계자 (Stakeholders)

- **채용 담당자 (Recruiter):** 대시보드를 통해 지원자들의 종합 리포트를 열람하고, 면접 질문지를 커스터마이징하는 관리자.
- **지원자 (Candidate):** 웹 브라우저를 통해 별도의 설치 없이 면접에 응시하고, 즉각적인 피드백을 수령하는 최종 사용자.
- **시스템 관리자/개발자:** AI 모델의 성능을 모니터링하고 인프라를 유지보수하는 기술 인력.

3. 시스템 요구사항 명세서 (SRS)

시스템 요구사항 명세서(SRS)는 개발될 소프트웨어의 기능과 제약 조건을 명확히 규정하는 문서입니다. 본 프로젝트는 생체 데이터를 다루고 실시간 상호작용이 필수적이므로, 기능적 요구사항뿐만 아니라 보안 및 성능과 같은 비기능적 요구사항의 정의가 매우 중요합니다.

3.1 기능적 요구사항 (Functional Requirements)

3.1.1 면접 시뮬레이션 엔진 (Interview Simulation Engine)

- REQ-F-001: 적응형 질문 생성 (Adaptive Questioning)
시스템은 정해진 질문 리스트를 순차적으로 제시하는 것을 넘어, 지원자의 답변 내용을 실시간으로 분석하여 꼬리 질문(Follow-up Questions)을 생성해야 합니다. 예를 들어, 지원자가 "MSA 경험이 있다"고 답변하면, "MSA 도입 시 겪었던 트랜잭션 관리의 어려움은 무엇이었나?"와 같은 구체적인 심층 질문을 생성해야 합니다. 이는 LangChain의 메모리 모듈과 RAG 기술을 결합하여 구현됩니다.
- REQ-F-002: 멀티모달 인터랙션 (Multimodal Interaction)
면접 과정은 음성 대화뿐만 아니라, 화상(Video), 텍스트(Chat/Code), 드로잉(Whiteboard) 등 다양한 입력을 동시에 처리해야 합니다. 지원자가 말하는 동안의 표정 변화와 목소리의 떨림 등을 감지하여 '자신감', '당황함' 등의 비언어적 지표를 추출해야 합니다.
- REQ-F-003: 실시간 개입 (Interruption Handling)
실제 면접과 유사하게, 지원자의 답변이 너무 길어지거나 주제를 벗어날 경우 AI 면접관이 정중하게 개입하여 다음 질문으로 넘어갈 수 있어야 합니다. 이를 위해 VAD(Voice Activity Detection)와 Turn-taking 알고리즘이 고도화되어야 합니다.

3.1.2 기술 역량 평가 (Technical Assessment)

- REQ-F-004: 라이브 코딩 환경
Python, JavaScript 등 주요 언어를 지원하는 웹 IDE가 내장되어야 하며, 지원자가 작성한 코드를 샌드박스 환경에서 실행하고 그 결과를 AI가 분석해야 합니다. 단순히 실행 결과의 정답 여부뿐만 아니라, 시간 복잡도, 코드 스타일, 주석 작성 여부 등을 종합적으로 평가해야 합니다.
- REQ-F-005: 시스템 설계 화이트보드
시스템 아키텍처 면접을 위해 다이어그램을 그릴 수 있는 캔버스를 제공하고, AI가 이를 시각적으로 인식(GPT-4V 등 활용)하여 아키텍처의 타당성을 평가해야 합니다.

3.1.3 결과 분석 및 리포팅 (Reporting)

- REQ-F-006: 상세 피드백 리포트
면접 종료 후, STAR 기법(Situation, Task, Action, Result)에 기반한 답변 구조 분석, 사용된 핵심 키워드, 발화 속도 및 발음의 명확성, 시선 처리 등을 포함한 종합 리포트를 생성해야 합니다.
- REQ-F-007: 채용 적합도 스코어링
사전에 정의된 루브릭(Rubric)에 따라 각 역량별 점수(1~5점 척도)를 산출하고, 채용 담당자가 직관적으로 판단할 수 있는 합격/불합격 추천 의견을 제시해야 합니다.

3.2 비기능적 요구사항 (Non-Functional Requirements)

3.2.1 성능 및 확장성 (Performance & Scalability)

- REQ-N-001: 초저지연 통신
WebRTC 기반의 영상 스트리밍과 음성 데이터 처리는 지연 시간을 최소화해야 합니다. 특히 STT(Speech-to-Text) 변환과 LLM 추론 시간을 포함한 전체 응답 지연은 1.5초를 넘지 않아야 사용자가 위화감을 느끼지 않습니다.
- REQ-N-002: 동시 접속 처리
채용 시즌에 급증하는 트래픽을 감당하기 위해, 백엔드 시스템은 수평적 확장(Horizontal Scaling)이 가능한 구조여야 합니다. Kubernetes 기반의 오케스트레이션을 통해 수백 개의 동시 면접 세션을 안정적으로 처리해야 합니다.

3.2.2 보안 및 규정 준수 (Security & Compliance)

- REQ-N-003: 생체 데이터 보호
면접 영상 및 음성 데이터는 개인정보보호법(GDPR, CCPA 등)에 따라 엄격하게 관리되어야 합니다. 모든 데이터는 전송 시(TLS 1.3) 및 저장 시(AES-256) 암호화되어야 하며, 사용자가 원할 경우 즉시 영구 삭제 가능한 기능을 제공해야 합니다.
- REQ-N-004: 공정성 및 편향 방지
AI 모델은 인종, 성별, 억양에 따른 편향이 없도록 다양한 데이터셋으로 검증되어야 하며, 평가 결과에 대한 설명 가능성(Explainability)을 제공해야 합니다.

4. 소프트웨어 아키텍처 설계서 (SAD)

프로젝트의 아키텍처는 고성능 실시간 처리를 위한 이벤트 기반 마이크로서비스(Event-Driven Microservices) 패턴을 채택합니다. 파이썬의 비동기 처리 능력을 극대화하기 위해 FastAPI를 핵심 프레임워크로 사용하며, 무거운 AI 연산은 별도의 워커(Worker)로 분리하여 처리합니다.

4.1 시스템 논리 아키텍처 (Logical Architecture)

시스템은 크게 클라이언트(Frontend), API 게이트웨이, 실시간 통신 서버, AI 처리 서비스, 데이터 스토리지로 구성됩니다.

계층 (Layer)	구성 요소 (Component)	기술 스택 (Tech Stack)	역할 및 책임
Presentation	Candidate App	React, Next.js, WebRTC API	면접 진행, 영상/음성 캡처, IDE 제공
	Recruiter Dashboard	React, Recharts	지원자 관리, 리포트 조회, 통계 시각화
Gateway	API Gateway	NGINX / Traefik	로드 밸런싱, SSL 종단, 라우팅
Application	Core API Service	FastAPI (Python)	사용자 인증, 세션 관리, 비즈니스 로직
	Signaling Server	FastAPI (WebSockets)	WebRTC 연결 수립 (SDP/ICE 교환)
Real-time	Media Server	Python (aiortc or GStreamer)	미디어 스트림 수신, 분기(Splitting), 녹화
AI Services	STT Service	Deepgram SDK / Whisper	실시간 음성 인식
	LLM Orchestrator	LangChain, LangGraph	대화 흐름 제어, 질문 생성, 답변

			평가
	Emotion Engine	DeepFace, Hume AI	표정 및 음성 감정 분석
Async Tasks	Task Queue	Celery	리포트 생성, 비디오 인코딩, 배치 분석
Data	Main DB	Oracle	사용자 정보, 면접 기록, 루브릭 데이터
	Vector DB	Pinecone / pgvector	질문 임베딩, RAG용 지식 베이스
	Cache/Broker	Redis	세션 상태 저장, 메시지 브로커
	Object Storage	Google Cloud Platform(GCP)	녹화 영상, 로그 파일 저장

4.2 상세 기술 선정 및 근거 (Technology Rationale)

4.2.1 웹 프레임워크: FastAPI vs Django

본 프로젝트에서는 **FastAPI**를 선정합니다.

- 비동기 처리(**Asynchronous I/O**): WebRTC 시그널링과 수많은 AI API 호출(OpenAI, Deepgram 등)은 I/O 바운드 작업입니다. FastAPI는 Python의 **asyncio**를 네이티브로 지원하여, Django(WSGI 기반) 대비 높은 동시성 처리 성능을 보장합니다.
- 데이터 검증: Pydantic을 이용한 강력한 타입 힌팅과 유효성 검사는 복잡한 AI 모델의 입출력 데이터(JSON)를 안정적으로 처리하는 데 필수적입니다.
- AI 친화성: 파이썬 기반의 AI 라이브러리(PyTorch, TensorFlow)와의 통합이 매끄러워, 별도의 모델 서빙 서버 없이도 경량화된 추론을 백엔드 내에서 수행하기 용이합니다.

4.2.2 실시간 통신: WebRTC 아키텍처

브라우저 간 직접 연결(P2P) 방식보다는 **SFU(Selective Forwarding Unit)** 또는 서버 사이드 프로세싱 구조를 채택합니다.

- 이유: AI가 면접관으로서 영상과 음성을 실시간으로 분석해야 하므로, 미디어 스트림이 서버를 거쳐야 합니다.
- 구현: 클라이언트의 WebRTC 스트림을 서버(Python aiortc 또는 GStreamer

파이프라인)에서 수신합니다. 수신된 오디오 스트림은 STT 엔진으로, 비디오 스트림은 컴퓨터 비전 모델(DeepFace)로 분기(Forking)되어 병렬 처리됩니다.

4.2.3 비동기 작업 처리: Celery + Redis

실시간성이 덜 중요한 작업(예: 면접 종료 후 종합 리포트 생성, 고해상도 영상 저장)은 Celery를 통해 비동기로 처리하여 API 서버의 부하를 줄입니다. Redis는 Celery의 브로커 역할뿐만 아니라, LLM의 대화 맥락(Context)을 저장하는 단기 메모리으로도 활용됩니다.

5. AI 및 데이터 엔지니어링 명세서

본 프로젝트의 핵심 경쟁력은 AI 모델의 정교한 오케스트레이션에 있습니다. 단순한 API 호출을 넘어, 각 모델이 유기적으로 협력하는 파이프라인을 설계해야 합니다.

5.1 대화형 AI 엔진 (LLM & RAG Strategy)

5.1.1 검색 증강 생성 (RAG) 아키텍처

일반적인 LLM은 특정 기업의 인재상이나 최신 기술 트렌드를 알지 못할 수 있습니다. 이를 보완하기 위해 RAG 아키텍처를 도입합니다.

- 데이터 수집 및 인덱싱:
 - 수천 개의 기술 면접 질문(알고리즘, 시스템 설계, 언어별 특성 등)과 행동 면접 질문(STAR 기법 기반)을 수집합니다.
 - 각 질문은 난이도, 주제, 직무, 평가 기준(Rubric)과 함께 메타데이터화되어 벡터 데이터베이스(Pinecone/pgvector)에 임베딩(Embedding) 형태로 저장됩니다.
- 검색(Retrieval) 전략:
 - 면접 진행 중 현재 대화의 맥락(Context)과 지원자의 이전 답변을 쿼리로 변환하여, 다음에 할 가장 적절한 질문을 벡터 DB에서 검색합니다.
 - 단순 유사도 검색뿐만 아니라, MMR(Maximum Marginal Relevance) 기법을 사용하여 질문의 다양성을 확보합니다.

5.1.2 프롬프트 엔지니어링 및 페르소나 설정

LLM이 일관된 면접관의 태도를 유지하도록 시스템 프롬프트를 정교하게 설계합니다.

- 시스템 프롬프트 예시: "당신은 15년 차 시니어 백엔드 개발자 면접관입니다. 지원자의 답변이 모호할 경우, 즉시 정답을 알려주지 말고 힌트를 주어 유도하십시오. 기술적인 용어 사용의 정확성을 엄격히 평가하십시오."
- 구조화된 출력(Structured Output): LLM의 응답은 단순 텍스트가 아닌 JSON 포맷으로 강제하여, 발화할 텍스트(TTS용)와 시스템 제어 신호(예: 코딩 테스트 화면으로 전환, 면접 종료 등)를 분리합니다.

5.2 음성 처리 파이프라인 (Speech Processing)

5.2.1 STT (Speech-to-Text) 선정: Deepgram Nova-2

- 비교 분석: OpenAI Whisper는 정확도는 높지만 기본적으로 배치(Batch) 처리 방식이라 실시간 대화에는 지연(Latency) 문제가 발생할 수 있습니다. 반면, Deepgram은 스트리밍에 최적화되어 있어 300ms 미만의 지연 시간을 보장하며, 중간 발화(Interim Results)를 제공하여 빠른 반응성을 구현할 수 있습니다.
- 구현 전략: WebSocket을 통해 들어오는 오디오 청크를 Deepgram 스트리밍 API로 전송하고, 반환되는 텍스트 스트림을 즉시 LLM으로 전달합니다.

5.2.2 TTS (Text-to-Speech) 및 Prosody

- **감정적 음성 합성:** Hume AI 또는 ElevenLabs와 같은 고품질 TTS 엔진을 사용하여, AI 면접관이 상황에 맞는 어조(격려하는 톤, 압박하는 톤 등)를 구사하도록 합니다.

5.3 감정 및 비언어 분석 (Emotion & Non-verbal Analysis)

5.3.1 얼굴 표정 분석 (Facial Expression)

- **도구:** DeepFace 라이브러리 (Python).
- **파이프라인:**
 1. 서버로 전송된 비디오 스트림에서 초당 1~2 프레임(FPS)을 추출합니다.
 2. DeepFace의 analyze() 함수를 호출하여 7가지 기본 감정(행복, 슬픔, 분노, 놀람, 공포, 혐오, 중립)의 확률 분포를 계산합니다.
 3. 이 데이터는 시계열 데이터베이스(RedisTimeSeries 등)에 저장되어, 면접 후 "어떤 질문에서 지원자가 당황했는지"를 타임라인 형태로 시각화하는 데 사용됩니다.

5.3.2 음성 운율 분석 (Audio Prosody)

- 단순한 텍스트 감정 분석을 넘어, 목소리의 높낮이(Pitch), 떨림(Jitter), 말하기 속도 등을 분석하여 지원자의 '자신감'과 '긴장도'를 측정합니다. 이는 librosa 라이브러리를 이용한 오디오 신호 처리 또는 Hume AI의 Prosody 모델을 통해 수행됩니다.¹³

6. API 및 데이터 설계서

6.1 데이터베이스 스키마 설계 (ERD Narrative)

관계형 데이터베이스(Oracle)는 데이터 무결성을 보장하기 위해 사용됩니다.

- **Users (사용자):** id, email, role (candidate/recruiter), password_hash, created_at.
- **Interviews (면접 세션):** id, candidate_id, job_posting_id, status (scheduled/live/completed), start_time, end_time, overall_score.
- **Questions (질문 은행):** id, content, category, difficulty, rubric_json (평가 기준), vector_id (Vector DB 참조).
- **Transcripts (대화 기록):** id, interview_id, speaker (AI/User), text, timestamp, sentiment_score.
- **Evaluation_Reports (평가 리포트):** id, interview_id, technical_score, communication_score, cultural_fit_score, summary_text, details_json.

6.2 핵심 API 명세 (Interface Contract)

시스템 간의 통신 규약은 명확해야 합니다. 다음은 핵심 기능에 대한 API 명세 예시입니다.

6.2.1 면접 세션 시작 (Start Session)

- **Endpoint:** POST /api/v1/interviews/{interview_id}/start
- **Request Header:** Authorization: Bearer {token}
- **Response (200 OK):**

```
JSON
{
  "session_token": "wss_token_xyz",
  "websocket_url": "wss://api.mockinterview.com/ws/interview/{interview_id}",
  "ice_servers": [
    { "urls": "stun:stun.l.google.com:19302" },
    { "urls": "turn:turn.mockinterview.com", "username": "user", "credential": "pwd" }
  ],
  "config": {
    "video_codec": "VP8",
    "audio_codec": "Opus"
  }
}
```

해설: 클라이언트가 WebRTC 연결을 수립하기 위해 필요한 ICE 서버 정보와 WebSocket URL을 반환합니다. TURN 서버 정보를 동적으로 제공하여 네트워크 보안을 강화합니다.

6.2.2 AI 평가 결과 저장 (Save Evaluation)

- **구조화된 출력 (JSON Schema):** LLM 에이전트가 생성하는 평가 데이터의 형식을

강제합니다.

JSON

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "technical_accuracy": {
      "type": "integer",
      "minimum": 1,
      "maximum": 5,
      "description": "기술적 답변의 정확도 점수"
    },
    "communication_clarity": {
      "type": "integer",
      "minimum": 1,
      "maximum": 5
    },
    "key_observations": {
      "type": "array",
      "items": { "type": "string" },
      "description": "면접관이 관찰한 주요 강점 및 약점"
    },
    "rubric_match": {
      "type": "object",
      "description": "사전 정의된 루브릭 항목별 달성 여부"
    }
  },
  "required": ["technical_accuracy", "key_observations"]
}
```

7. 작업 분류 체계 (WBS) 및 프로젝트 일정

성공적인 프로젝트 관리를 위해 전체 작업을 7개의 주요 단계(Phase)로 세분화합니다.

Phase 1: 기획 및 요구사항 분석 (Weeks 1)

- **1.1 이해관계자 인터뷰:** HR 전문가 인터뷰를 통해 면접 평가 루브릭 정의.
- **1.2 SRS 작성 및 승인:** 본 문서의 내용을 바탕으로 요구사항 확정.
- **1.3 기술 검증 (PoC):** Python 환경에서의 WebRTC 스트리밍 및 Deepgram STT 연동 테스트.

Phase 2: 시스템 아키텍처 및 인프라 구축 (Weeks 2)

- **2.1 클라우드 환경 구성:** AWS/GCP VPC 설정, EKS(Kubernetes) 클러스터 프로비저닝.
- **2.2 CI/CD 파이프라인 구축:** GitHub Actions를 이용한 자동 빌드 및 배포 구성.
- **2.3 데이터베이스 구축:** PostgreSQL 및 Redis 클러스터 설정, 마이그레이션 스크립트 작성.

Phase 3: 백엔드 코어 개발 (Weeks 3)

- **3.1 FastAPI 기본 구조 구현:** 인증(Auth), 사용자 관리 API.
- **3.2 WebRTC 시그널링 서버 개발:** WebSocket 핸들러 구현.
- **3.3 미디어 서버 파이프라인:** aiortc 기반의 오디오/비디오 스트림 수신 및 분기 처리 로직.

Phase 4: AI 모델 통합 및 에이전트 개발 (Weeks 4)

- **4.1 RAG 시스템 구축:** 질문 데이터셋 수집, 벡터 DB 인덱싱, Retriever 구현.
- **4.2 LLM 에이전트 개발:** LangChain을 이용한 면접관 페르소나 구현, 대화 상태 관리(Memory) 로직.
- **4.3 감정 분석 워커 개발:** DeepFace 및 오디오 분석 모듈을 Celery 워커로 구현 및 최적화.

Phase 5: 프론트엔드 개발 (Weeks 5, 병렬 진행)

- **5.1 면접장 UI 개발:** 화상 채팅, 코드 에디터(Monaco Editor), 화이트보드 컴포넌트 구현.
- **5.2 대시보드 개발:** 채용 담당자용 관리자 페이지, 차트 시각화 구현.

Phase 6: 통합 테스트 및 QA (Weeks 6)

- **6.1 부하 테스트 (Load Testing):** Locust를 사용하여 동시 접속 500명 상황 시뮬레이션 및 병목 구간 튜닝.
- **6.2 AI 편향성 테스트:** 다양한 성별, 인종, 억양의 테스트 케이스를 수행하여 평가 공정성 검증.
- **6.3 사용자 수용 테스트 (UAT):** 실제 채용 담당자 및 구직자 그룹을 대상으로 베타 테스트 진행.

Phase 7: 배포 및 운영 이관 (Week 7)

- **7.1 프로덕션 배포:** Blue/Green 배포 전략을 통한 무중단 오픈.
- **7.2 모니터링 대시보드 설정:** Prometheus/Grafana를 통한 시스템 리소스 및 AI 토큰 사용량 모니터링.

8. 품질 보증(QA) 및 평가 계획

AI 시스템의 평가는 일반적인 소프트웨어 테스트와 다릅니다. 기능적 오류뿐만 아니라 AI의 판단 품질을 지속적으로 검증해야 합니다.

8.1 평가 루브릭 (Evaluation Rubric) 설계

객관적인 평가를 위해 구조화된 루브릭을 시스템에 내재화합니다.

평가 항목	1점 (미흡)	3점 (보통)	5점 (탁월)
개념 이해도	핵심 개념을 정의하지 못하거나 오개념을 가지고 있음.	개념은 정확히 정의하나, 실무 적용 사례를 제시하지 못함.	개념을 정확히 이해하고, 장단점(Trade-off)과 실제 적용 사례를 논리적으로 설명함.
문제 해결력	힌트를 주어도 문제 해결의 실마리를 찾지 못함.	힌트를 통해 문제를 해결하나, 최적화된 방법은 아님.	문제를 스스로 정의하고 다양한 해결책을 비교 분석하여 최적해를 도출함.
커뮤니케이션	답변이 장황하고 핵심이 불분명하며, 비언어적 태도가 소극적임.	질문의 의도를 파악하고 적절히 대답하나, 구조화가 부족함.	두괄식으로 명확히 답변하며, 자신감 있는 태도와 적절한 시선 처리를 유지함.

8.2 AI 모델 성능 지표

- **STT 정확도 (WER):** 음성 인식 오류율(Word Error Rate)을 10% 미만으로 유지.
- **감정 분석 정확도:** 표준 데이터셋(FER-2013 등) 기준 80% 이상의 분류 정확도 확보.
- **응답 적절성:** LLM이 생성한 평가 코멘트와 인간 면접관의 평가 간의 일치도(Human-AI Agreement)를 주기적으로 측정.

9. 결론 및 제언

프로젝트는 파이썬의 강력한 AI 생태계와 현대적인 웹 기술을 결합하여, 채용 프로세스의 혁신을 이끌어낼 잠재력을 가지고 있습니다. **FastAPI**와 **WebRTC**를 통한 기술적 견고함, **LangChain**과 **RAG**를 통한 지능적 유연성, 그리고 정교한 루브릭을 통한 평가의 공정성이 본 플랫폼의 성공을 좌우할 것입니다.

제안된 아키텍처와 **WBS**에 따라 프로젝트를 수행함으로써, 귀사는 단순한 면접 도구를 넘어선 '지능형 채용 어시스턴트'를 확보하게 될 것입니다. 향후 발전 방향으로서는 멀티턴(Multi-turn) 대화의 문맥 유지 능력을 강화하고, 기업별 커스텀 LLM 파인튜닝(Fine-tuning) 기능을 도입하여 플랫폼의 가치를 더욱 높일 것을 제언합니다.

10. 부록: 문서 체크리스트

본 프로젝트 착수 시 반드시 구비되어야 할 문서 목록입니다.

1. ☐ 프로젝트 헌장 (Project Charter)
2. ☐ 요구사항 정의서 (SRS) v1.0
3. ☐ 시스템 아키텍처 설계서 (SAD)
4. ☐ API 명세서 (Swagger/OpenAPI)
5. ☐ 데이터베이스 ERD (Entity Relationship Diagram)
6. ☐ UI/UX 와이어프레임 및 디자인 가이드
7. ☐ AI 모델 검증 보고서 (Model Card)
8. ☐ 테스트 계획서 (Test Plan) 및 테스트 케이스
9. ☐ 개인정보 처리방침 및 보안 감사 보고서
10. ☐ 사용자 매뉴얼 (User/Admin Guide)

(이하 생략된 상세 내용은 각 섹션의 하위 문서로 별도 첨부될 수 있음)